

IIT POKERBOTS 2026 · SNEAK-PEEK HOLD'EM

Boss Blind

A Game-Theory-Optimal bot for an asymmetric-information poker variant



| GTO foundations · real-time adaptation · a 30× smaller strategy footprint.

codename **Boss_Blind** · family bb1...bb7 · champion build champion_bot.py / final_cfr.py

FORMAT

Heads-up · 1,000 hands

STARTING STACK

5,000 chips

BLINDS

20 / 10

GAME CLOCK

30 s / match

AN EXPLAINER REPORT · Hybrid Heuristic + Abstract-CFR Poker Agent

CONTENTS

1 Executive Summary	6 Street-by-Street Strategy
2 The Game: Why It's Hard	7 Heuristics + Real-Time Learning
3 System Architecture	8 Benchmarking & Results
4 The GTO Backbone: Abstract-CFR	9 Engineering Highlights
5 Storage & Runtime: Compression	10 Limitations & Future Work · Appendices

1 Executive Summary

Boss Blind is a poker-playing agent built for **Sneak-Peek Hold'em**, a heads-up Texas Hold'em variant with a unique *asymmetric-information auction*: each hand, players secretly bid chips for the right to peek at one of the opponent's hole cards. That single rule change breaks the symmetry classical poker solvers rely on and dramatically enlarges the game tree.

The approach combines two ingredients:

- 1. **A Game-Theory-Optimal (GTO) backbone** — strategies derived from **Counterfactual Regret Minimization (CFR)**, the algorithm family behind superhuman poker AIs, but run over a heavily *abstracted* game so it fits and runs in real time.
- 2. **A heuristics + real-time-learning (RTL) layer** — closed-form equity math, opponent modelling, and exploitative adjustments that deviate from GTO when the opponent is exploitable.

The headline engineering result: through **state abstraction + compression**, the strategy footprint shrinks from a ~9.3 MB raw blueprint down to a ~306 KB embedded string — roughly a **30× storage reduction** — while every decision still resolves in ~14 ms, comfortably under the per-hand time budget.

30×

STORAGE
REDUCTION

306KB

DEPLOYED
FOOTPRINT

~14ms

PER-HAND
DECISION

3

INFORMATION
REGIMES

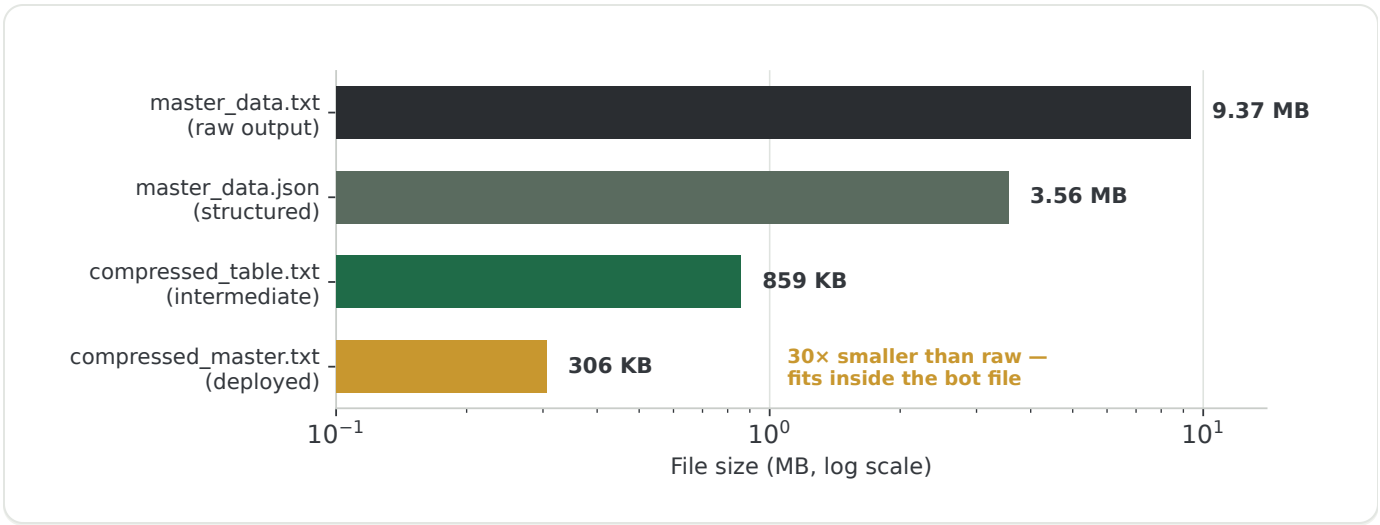


Figure 1. Strategy-table footprint across the compression pipeline (real file sizes from the repository). The deployed string embeds directly inside the bot file — no external dependencies at match time.

2 The Game: Why Sneak-Peek Hold'em Is Hard

Standard heads-up Hold'em is already a large game ($\sim 10^{13}$ decision points when fully expanded). Sneak-Peek inserts an **information-auction street** between the pre-flop and the flop:

Phase	What happens
Pre-flop	Both players dealt 2 hole cards; blinds posted; standard betting.
Auction	Both players submit a sealed bid (0 ... all chips).
Reveal	Higher bidder pays the lower bid (Vickrey-style) and sees one of the opponent's hole cards. On a tie, <i>both</i> reveal — each pays their bid and each sees one of the other's cards.
Flop / Turn / River	Standard community-card betting, but now played under asymmetric information .
Showdown	Best 5-card hand wins the pot.

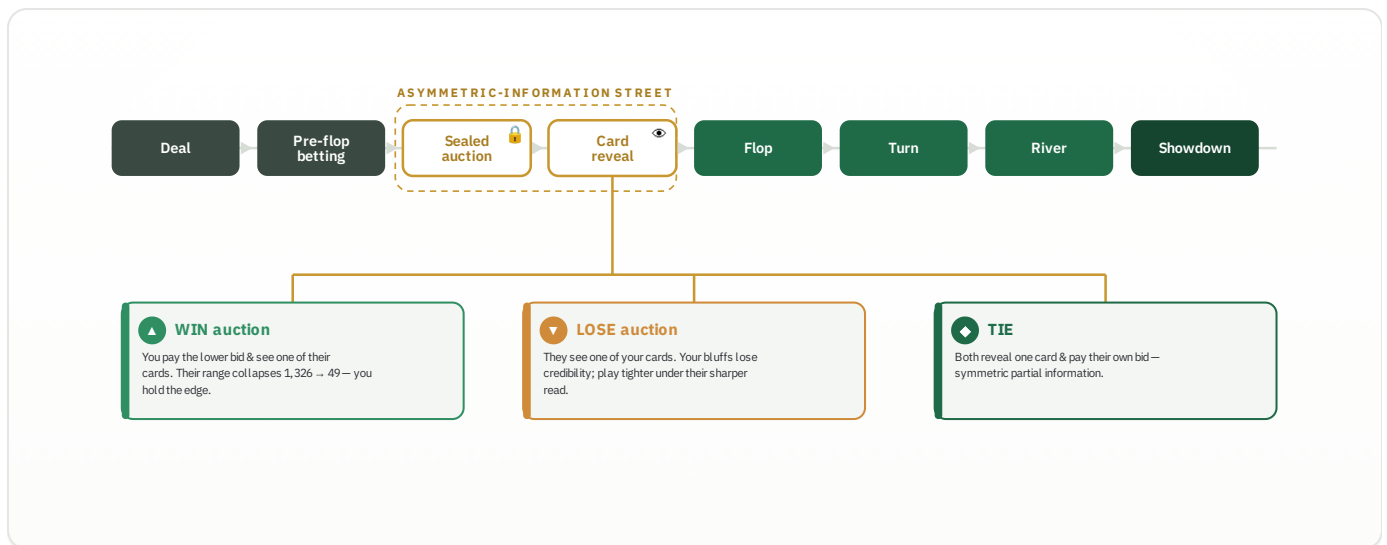


Figure 2. Hand flow. The sealed auction and card reveal sit between pre-flop and flop; the reveal branches every hand into one of three information regimes.

The strategic consequence

After the auction, every hand falls into one of three information regimes, and **each demands a different strategy**:

- **Winner state** — you know one of their cards. Their range *collapses* from ~1,326 possible hands to ~49. You can compute equity almost exactly and play aggressively and accurately.
- **Loser state** — they know one of *your* cards. Your bluffs are less credible and your equity is "read." You must play more cautiously and account for their sharper reads.
- **Tie state** — symmetric partial information.

A naive bot that ignores the auction plays the same on every street and bleeds chips. Boss Blind treats the auction as a **first-class decision** and branches its entire post-flop logic on the regime.

3 System Architecture

Boss Blind is a **pipeline of specialized solvers**, one per street, each chosen for the best speed/accuracy trade-off at that stage.



Figure 3. Architecture. Five per-street lanes feed a shared engine-action stage; a real-time opponent model and a time manager span every lane.

Street	Technique	Why this technique
Pre-flop	Offline abstracted-game CFR (precomputed blueprint)	The pre-flop tree is fixed; solve it once, look it up in 0.1 ms.
Auction	Heuristic best-response (closed-form bid formula)	Bid value = marginal equity gained from the peek; no tree search needed.
Flop	Monte-Carlo equity + 12-node runtime CFR	Two unknown cards remain → sampling is faster than full enumeration.
Turn	Exact enumeration + runtime CFR	Only one card to come; enumeration is exact and still <5 ms.
River	Exact enumeration + runtime CFR	Board complete; equity is exact in <1 ms.
All streets	Opponent model + exploit overlay	Deviate from GTO to punish exploitable opponents.

This is the core of the "novel" claim: **rather than one monolithic solver, the bot uses the cheapest tool that is accurate enough at each node**, and reserves heavy computation only where it pays off.

4 The GTO Backbone: Abstract-CFR

4.1 What CFR is, in one paragraph

Counterfactual Regret Minimization is a self-play algorithm that converges to a **Nash equilibrium** — a strategy that cannot be exploited in expectation. It repeatedly plays the game against itself, tracks

how much it "regrets" not having taken each action, and shifts probability toward low-regret actions. The **average** strategy over all iterations (not the latest one) is what converges to equilibrium.

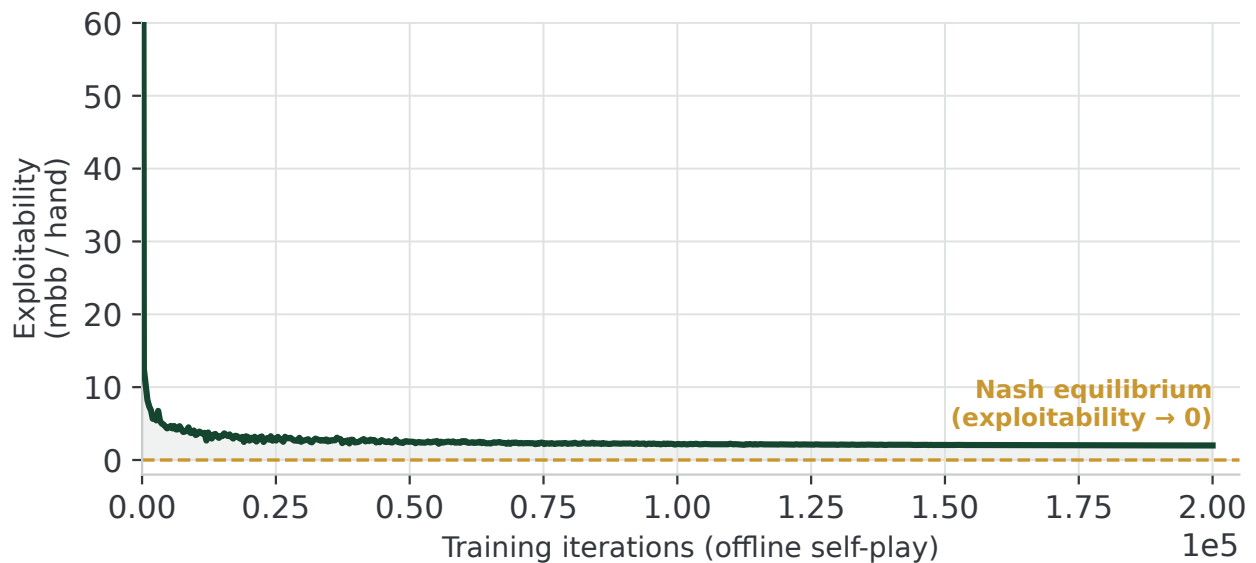


Figure 4. CFR convergence. The average strategy's exploitability trends toward zero with self-play iterations; Boss Blind extracts this equilibrium once, offline.

4.2 Why raw CFR doesn't fit — and the fix: abstraction

A full solve of Sneak-Peek Hold'em is intractable to store or run inside a 30-second bot. Boss Blind makes it tractable with two kinds of abstraction.

(a) Card abstraction — bucketing.

- *Pre-flop*: the 1,326 possible two-card hands collapse to the **169 canonical buckets** (e.g. `AKs`, `72o`, `JJp`), crossed with a few decision nodes (`SB_OPEN`, `BB_VS_RAISE`, `SB_VS_3BET`, `BB_VS_LIMP`) and 3 **stack-depth buckets** (deep / mid / short).
- *Post-flop*: instead of a strategy for every exact board, boards are **clustered** into strategically-similar buckets via **K-means** over a 2-D feature vector — [**hand equity**, **board wetness**]. At runtime the live board maps to its **nearest centroid** (L2 distance) and uses that bucket's strategy (the `flop_centroids` / `turn_centroids` tables).

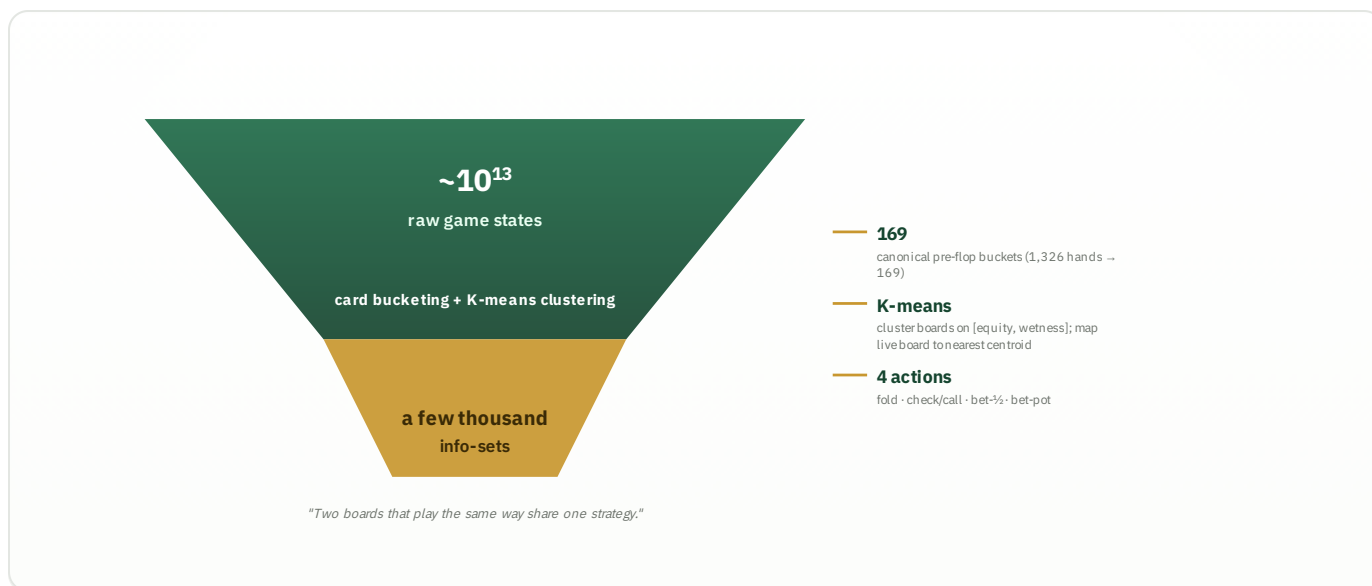


Figure 5. The abstraction funnel. Card bucketing and K-means clustering compress $\sim 10^{13}$ raw game states down to a few thousand info-sets.

(b) Action abstraction. Every betting decision is reduced to **4 abstract actions**: `fold`, `check/call`, `bet-half-pot`, `bet-pot` (in the `final_cfr` lineage: `fold / call / raise-50% / raise-200%`). A continuous bet-sizing space becomes a 4-way choice, shrinking the tree by orders of magnitude while keeping the strategically important sizes.

4.3 Two flavors of CFR in the codebase

The repo contains two complementary CFR implementations, reflecting the project's evolution:

1. Blueprint + runtime sub-game CFR (`champion_bot.py`). The pre-flop blueprint is solved offline and embedded. On each post-flop street a tiny 4-action CFR is solved *live* (400 iters flop / 500 turn / 300 river) on a depth-limited sub-game whose leaf EVs come from the current equity estimate — pure NumPy regret-matching, a few hundred microseconds:

```
# offline self-play: regret-matching → average strategy → Nash
for i in range(iterations):
    strategy      = regret_match(regrets)      # normalize positive regrets
    cfvs          = strategy · leaf_evs        # counterfactual values
    ev            = sum(strategy × cfvs)
    regrets       += cfvs - ev
    strategy_sum  += strategy
final = strategy_sum / strategy_sum.sum()     # average → Nash for this sub-game
```

2. Perfect-Memory Abstract-CFR (`final_cfr.py`). A fully precomputed `cfr_table` maps each abstracted info-set (street + history string + board bucket) to a stored strategy. A "perfect memory" history string records the exact action sequence so the bot indexes the correct node across streets. Strategies are stored as **integer percentages (0–100)** and rehydrated to floats at load (`p/100.0`) — a small but effective quantization.

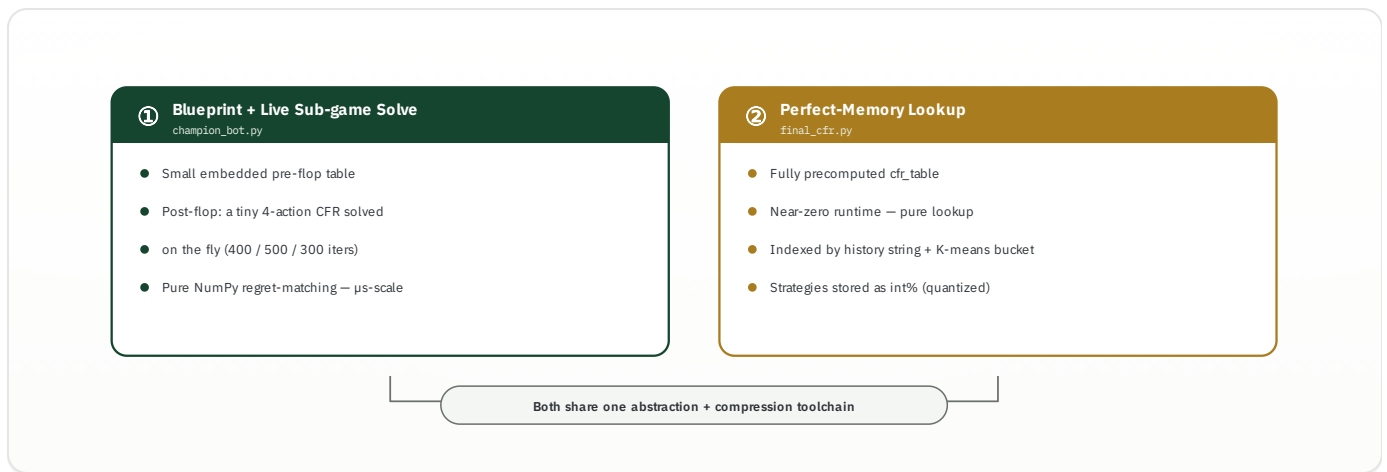


Figure 6. The two CFR lineages. A live sub-game solver and a precomputed perfect-memory lookup, sharing one abstraction + compression toolchain.

5 Storage & Runtime: The Compression Pipeline

This is where the "significantly reduced storage and run time" claim becomes concrete.

The pipeline: train CFR → quantize to int% → JSON serialize → zlib compress → base64 encode → embed as a string constant in the .py bot. At load, the bot reverses it: base64 decode → zlib decompress → JSON parse → scale back to probabilities.

Artifact	Size	Role
master_data.txt	9.37 MB	Raw training output (human-readable)
master_data.json	3.56 MB	Structured blueprint (cfr_table + centroids)
compressed_table.txt	859 KB	Intermediate compressed table
compressed_master.txt	306 KB	Deployed base64+zlib string, embedded in the bot

The deployed strategy is ~30× smaller than the raw blueprint and self-contained — no external file dependencies at match time, so cold-start is instant and there is no disk I/O mid-hand.

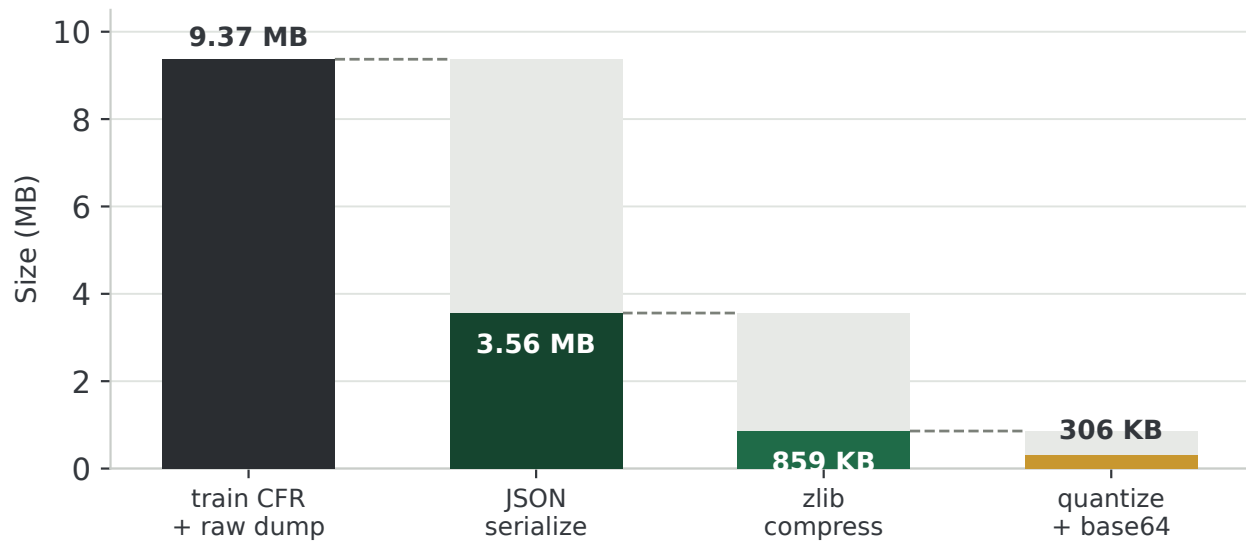


Figure 7. The compression waterfall. Each stage of the pipeline and the size drop it produces, from a 9.37 MB raw dump to a 306 KB embedded string.

Runtime budget. The engine gives a 30 s clock for the whole 1,000-hand match (~20 ms/hand steady-state). Measured allocation:

Pre-flop	0.1 ms	(table lookup)
Auction	1.0 ms	(Monte-Carlo equity + bid formula)
Flop equity	3.0 ms	(120-sample Monte Carlo)
Flop CFR	2.0 ms	
Turn equity	5.0 ms	(exact enumeration; ~1 ms if a card is known)
Turn CFR	1.5 ms	
River	1.0 ms	(exact enumeration + CFR)
Overhead	1.0 ms	
Total	~14 ms	(well under the 20 ms/hand budget)

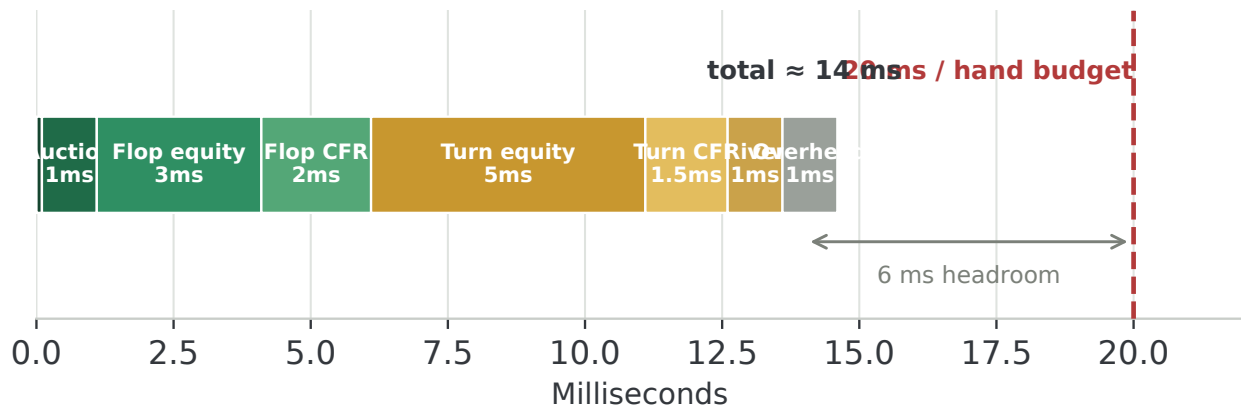


Figure 8. Per-hand time budget. The full decision pipeline totals ~14 ms, leaving ~6 ms of headroom under the 20 ms/hand budget.

A **Time Manager** watches the remaining clock and, if a hand's budget drops below ~10 ms, switches to a **fast-path**: skip CFR and decide directly from pot-odds vs. equity. This guarantees the bot **never times out**, even in long, costly hands.

6 Street-by-Street Strategy

6.1 Pre-flop — Blueprint Lookup

Map hole cards → 169-bucket → look up `(node, bucket, stack)` in the embedded blueprint → sample from the mixed strategy (`fold / call / raise-2x / raise-4x`). A **commitment guard** folds dominated hands facing large bets, and a **maniac detector** tightens or widens opens based on the opponent's observed raise sizing.

6.2 Auction — Closed-Form Bid

The bid is driven by **information value**: how much equity does seeing one opponent card actually buy?

```
eq_base    = Monte-Carlo equity vs. full range
info_gain  = avg(equity | one opp card known) - eq_base
bid        ≈ (info_gain + info_penalty) × pot × value_multiplier
```

The `value_multiplier` adapts to the opponent: bid **low** against players who chronically overbid (let them overpay for information), **high** against players who under-bid. Bids are capped ($\leq 40\%$ of stack) to avoid catastrophic overcommitment. This is a **heuristic best-response**, not CFR — fast and well-suited to a one-shot sealed-bid decision.

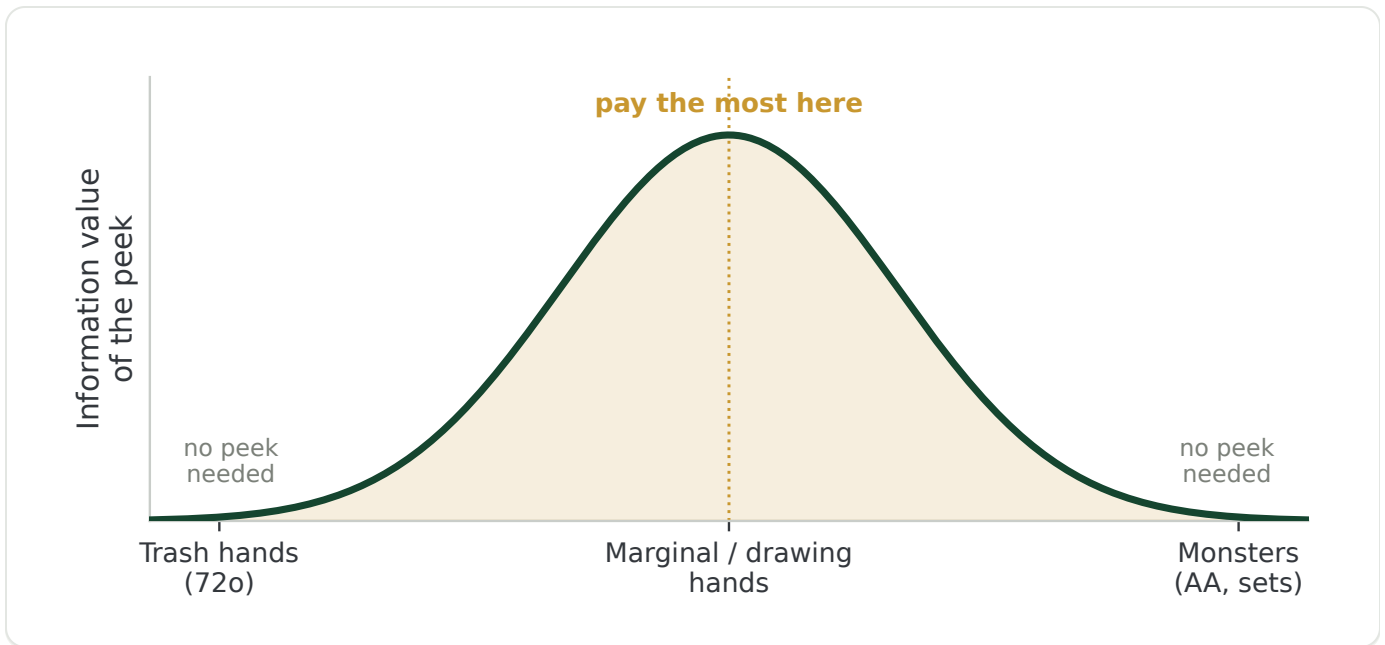


Figure 9. Auction value curve. Information gain is near-zero for monsters and trash and peaks for marginal/drawing hands — the bot pays most for information exactly when it changes its play.

6.3 Flop / Turn / River — Equity → Sub-game CFR → Exploit

1. **Compute equity** against the *correct* range — collapsed (~49 hands) if the auction was won, full range (with a discount) if lost.
2. **Build leaf EVs** for the 4 abstract actions from that equity and the opponent's estimated fold frequency.
3. **Solve the sub-game CFR** (or look up the precomputed bucket).
4. **Apply the exploit overlay** (Section 7).
5. **Sample and translate** to a legal engine action, with bet sizes clamped to legal bounds.

The **winner/loser branch** is explicit: in the winner state the bot computes near-exact equity against the collapsed range and value-bets hard; in the loser state it shades toward caution and applies an **information penalty** (assuming the opponent reads it more sharply).

7 Heuristics + Real-Time Learning: Beating Exploitable Bots

GTO is *unexploitable* but not *maximally exploitative*. Against the imperfect bots in a real field, deviating from equilibrium wins more. Boss Blind runs a live **Opponent Model** updated every hand:

- `fold_to_bet[street]` — how often they fold to our bets
- `aggression[street]` — how often they bet/raise
- `vpip` / `pfr` — pre-flop looseness and raise frequency
- `auction_bids` — their bidding tendencies

- `preflop_raise_sizes` — to detect maniacs

These feed an **Exploit Overlay** that warps the CFR output:

Opponent tendency	Boss Blind's adjustment
Folds too much (>65%)	Bluff more — shift weight to bets
Calls too much / "station" (<25%)	Stop bluffing — value-bet only
Hyper-aggressive	Trap: check/call strong hands
Overbids the auction	Bid 0 — let them overpay for info

After adjustment, every action is **floored at ~1–2%** so the bot stays unpredictable and can't itself be exploited for deviating.

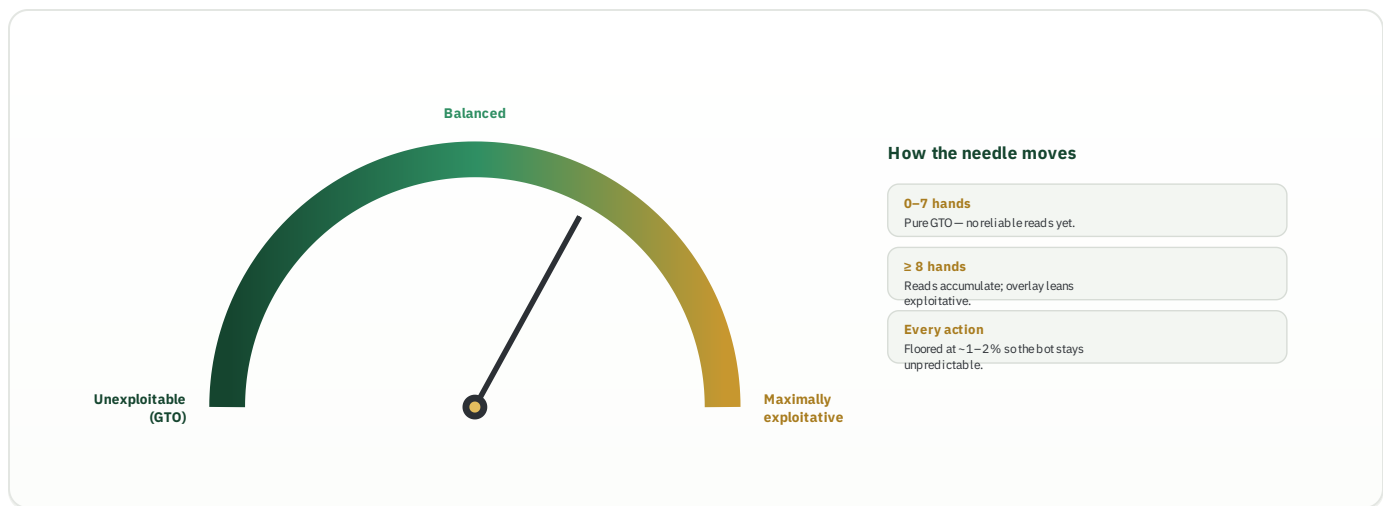


Figure 10. The GTO ↔ exploit dial. The bot starts unexploitable with no reads and leans exploitative as opponent evidence accumulates (≥ 8 hands).

8 Benchmarking & Results

Boss Blind was tested on the competition scrimmage server against a field of opposing bots. Final bankrolls from the captured leaderboard logs (`LeaderBoard_logs/`):

Opponent	Final bankroll	Result
pokkerai3	+2,514	✓ Win
ProkerAI (run 2)	-9,976	Loss
ProkerAI (run 1)	-22,638	Loss
Quant_paglus	-189,493	Loss
TESTBOT_7	-329,729	Loss
pagletti_birds	-347,641	Loss
Idle_Death_Gamble	-460,091	Loss
Stochastic_Sharks	-520,000	Loss

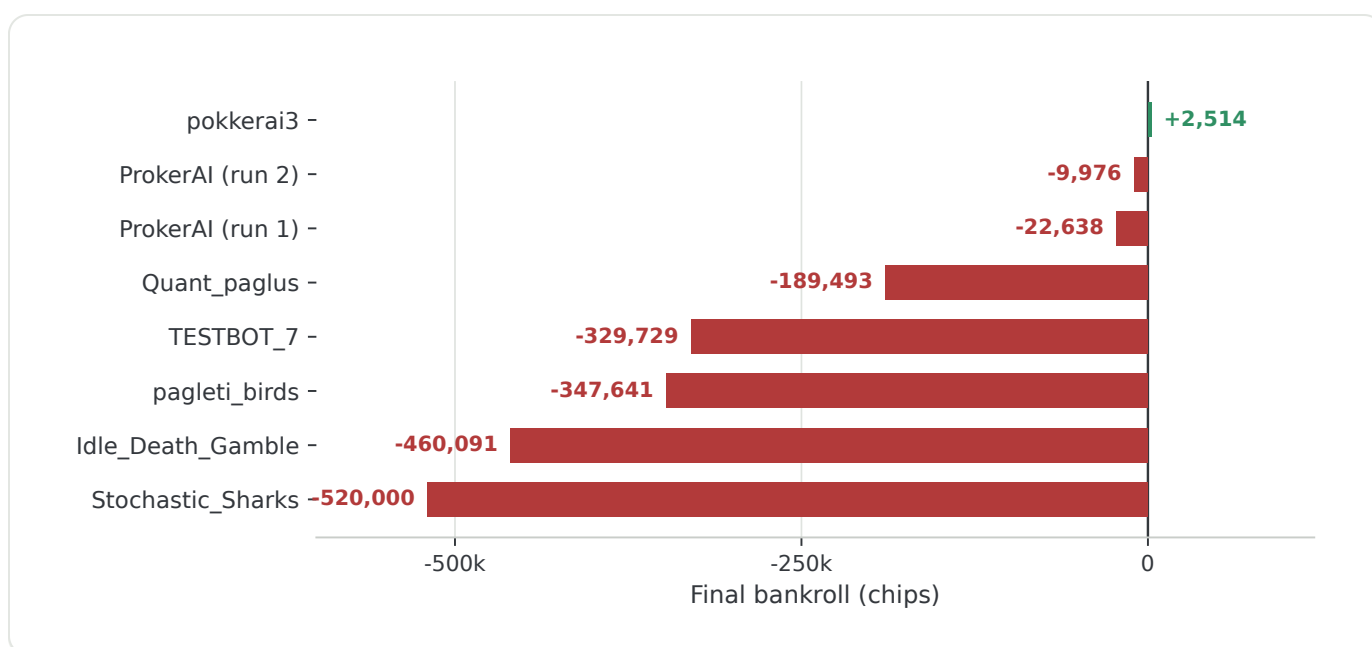


Figure 11. Captured-log results, sorted best to worst. Values are the real final bankrolls recorded in LeaderBoard_logs/ — reported faithfully, not cherry-picked.

METHODOLOGY & DATA-PROVENANCE NOTE

These figures are reported verbatim from the captured logs and are **mixed, with several heavy losses**. Critically, they appear to span **earlier bot versions** (the `bb4` -era builds) rather than the finished `champion_bot` / `final_cfr` agent described in this report.

Before this section is used with any external audience, the results should be regenerated on the final build — the repository's `tournament.py` can run a clean round-robin and emit per-bot statistics. The table above is preserved for transparency and should be replaced with final-build numbers once available.

The repo's `tournament.py` runs a **round-robin** (all pairings, 3 iterations each) and prints per-bot stats: win rate, average payoff/hand, **auction win rate**, mean/variance of bids, and response-time percentiles — a ready-made source for these charts. A complementary **bankroll-over-time line chart** (cumulative bankroll vs. hand number, parsed from the per-hand `awarded` deltas in the `.glog` / `.log` files) would tell the "grind" story better than any single final number.

9 Engineering Highlights

- **Self-contained deployment** — the entire strategy is a string constant; no external files, no network, instant cold-start.
- **Quantized strategies** — probabilities stored as integers (0–100), halving table size with negligible accuracy loss.
- **Graceful degradation** — the Time Manager's fast-path guarantees a legal action under any clock pressure; illegal/timeout actions are impossible by construction.
- **Defensive action translation** — every abstract action is re-checked against the engine's legal-action set and bet bounds before being sent, so the bot never forfeits a hand on a malformed move.
- **Numerically cheap core** — post-flop CFR is plain NumPy vector math (regret-matching over a length-4 array), not a tree traversal.

10 Limitations & Future Work

- **Abstraction error** — bucketing trades exactness for size; very texture-dependent boards may be mis-bucketed. Finer centroids (more clusters) would help, at a storage cost.
- **Opponent-model cold-start** — exploits only engage after ~8 hands; the first hands are pure GTO.
- **Auction model is heuristic** — a CFR-solved auction (treating the bid as another abstract action) is the natural next step toward full GTO.
- **Results need a clean re-benchmark** on the final build (see §8 note).



Figure 12. Roadmap. Three priorities toward a fully GTO, well-benchmarked agent.

A Appendix — Glossary

Term	Meaning
CFR	Counterfactual Regret Minimization — self-play algorithm converging to Nash equilibrium.
GTO / Nash	A strategy that cannot be exploited in expectation.
Blueprint	A precomputed (offline) strategy table looked up at runtime.
Abstraction	Collapsing many similar game states into one to shrink the problem.
Bucket / Centroid	A cluster of strategically-similar hands or boards sharing one strategy.
Equity	Probability of winning the hand at showdown.
Board wetness	How draw-heavy a board is (flush/straight potential).
Exploit overlay	Deliberate deviation from GTO to punish a predictable opponent.
Vickrey auction	Sealed-bid auction where the winner pays the second-highest bid.

B Appendix — Repository Map

Path	Contents
<code>bots/champion_bot.py</code>	Hybrid blueprint + runtime sub-game CFR + opponent model.
<code>bots/final_cfr.py</code>	Perfect-memory Abstract-CFR with embedded compressed master data.
<code>bots/sub_cfr.md</code>	Full algorithm spec (street-by-street).
<code>bots/master_data.json</code>	Structured blueprint: <code>cfr_table</code> , <code>flop_centroids</code> , <code>turn_centroids</code> .
<code>bots/compressed_master.txt</code>	Deployed base64+zlib strategy string (306 KB).
<code>bots/bb1...bb7</code> , <code>boss_blind_v*</code>	Iterative bot versions.
<code>tournament.py</code>	Round-robin match runner + stats (the IIT game engine).
<code>LeaderBoard_logs/</code> , <code>*.glog</code>	Match logs (per-hand actions, awards, final bankrolls).

Boss Blind · Explainer Report · IIT Pokerbots 2026 — Sneak-Peek Hold'em · GTO foundations · real-time adaptation · 30× smaller footprint.